

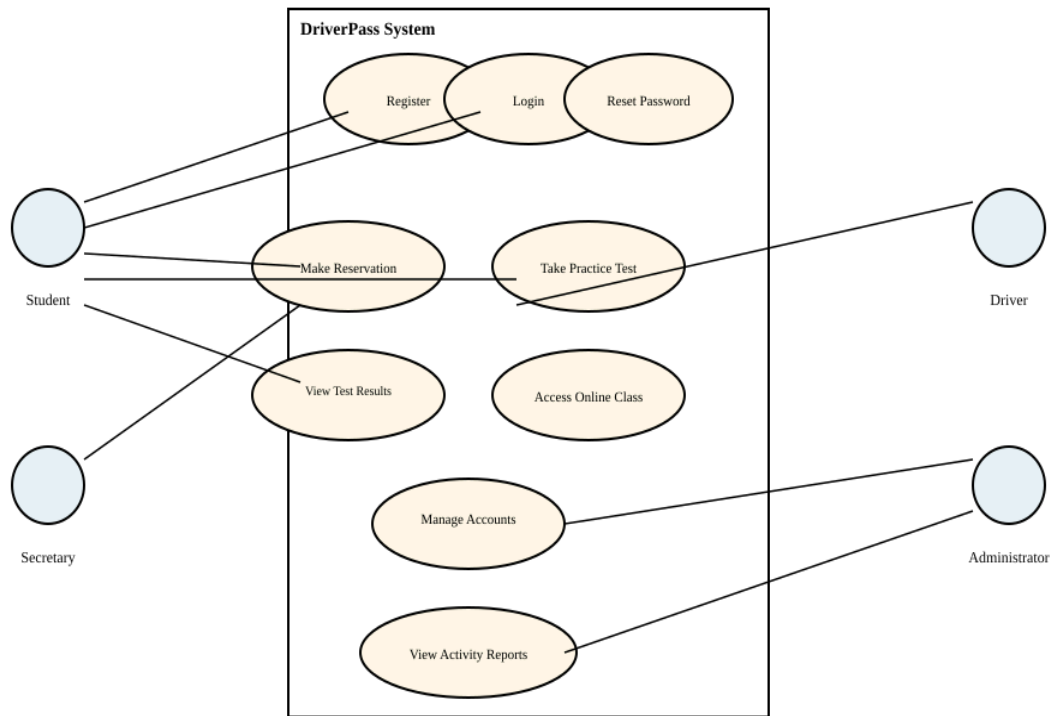
DriverPass System Design Document

System Analysis and Design

1. UML Use Case Diagram

The use case diagram identifies all system actors and their interactions with the DriverPass platform.

UML Use Case Diagram - DriverPass System



Key Actors:

Student: Register, login, make/modify/cancel reservations, take tests, access online classes

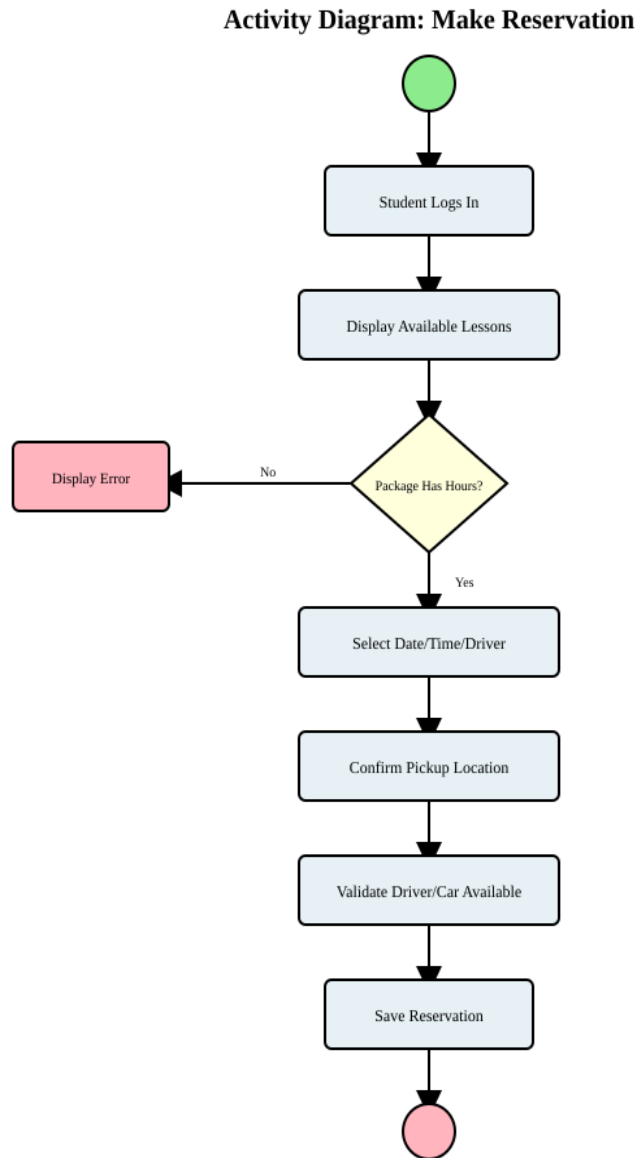
Secretary: Register students, schedule/modify appointments, view reservations

Driver: Login, view assigned lessons, add lesson notes

Administrator: Manage accounts, reset passwords, block accounts, view reports

2. UML Activity Diagram: Make Reservation

This diagram shows the flow when a student makes a driving lesson reservation, including validation and error handling.



Flow Summary: Student logs in → Views available lessons → System validates package hours → Student selects date/time/driver → System validates availability → Saves reservation with audit trail → Displays confirmation

3. UML Sequence Diagram

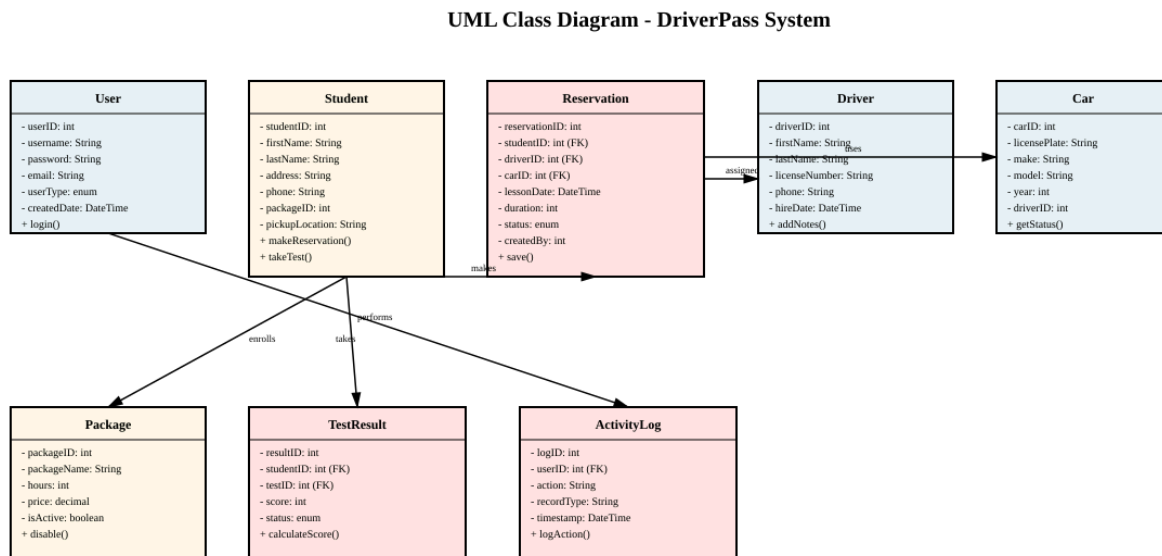
Interaction: Student → Browser → DriverPass System → Database → Payment Gateway

Key Messages:

1. Login request with credentials
2. System validates in database
3. Display available lesson options
4. Student selects lesson and submits reservation
5. System processes payment
6. System saves reservation with createdBy/lastModifiedBy audit trail
7. Confirmation sent to browser

4. UML Class Diagram

The class diagram defines the core data model for the DriverPass system with proper relationships and attributes.



Core Classes: User (base), Student, Reservation, Driver, Car, Package, TestResult, ActivityLog

Key Relationships: Student enrolls in Package → makes Reservations → assigned Driver (uses Car) → takes TestResults; all actions logged in ActivityLog with user and timestamp

5. Technical Requirements

Infrastructure

- Cloud platform (AWS/Azure/GCP) with managed backups and 99.5% uptime SLA
- Enterprise-grade web and database servers with redundancy
- HTTPS/TLS encryption for all data in transit

Software Stack

- Backend: Java Spring Boot or Node.js Express
- Frontend: React.js or Vue.js (responsive)
- Database: PostgreSQL or MySQL with indexed queries
- Payment Processing: Stripe or Square (PCI DSS compliant)

Security

- Role-based access control (RBAC) by user type
- Encrypted password storage (bcrypt/Argon2)
- Complete audit trail logging all user actions with timestamps
- Admin capability to reset passwords and block accounts

Performance

- Page load < 3 seconds (90th percentile)
- Support 500+ concurrent users
- Database query response < 200ms

6. Project Gantt Chart

